36. ArrayDeque in Java — Quick Notes

What is ArrayDeque?

- `ArrayDeque` (Array Double Ended Queue) implements the **Deque** interface.
- It can be used as: **Stack, Queue, Deque**
- Internally implemented using a **resizable circular array**.
- Generally **faster than LinkedList** because it is **cache-friendly**.

Hierarchy

Collection
↑
Queue
↑
Deque
↑
ArrayDeque

Important Notes

Topic	Details
Implements	`Deque<E>`
Deque extends	`Queue<E>`
Queue extends	`Collection<E>`
Preferred over Stack	Yes (single-threaded environment)
Preferred over LinkedList (Queue/Deque)	Yes
Internal Structure	Resizable Circular Array
Thread Safe	No
Null Elements Allowed	No
Random Access (`get(index)`)	Not Supported
`set(index)`	Not Supported

Import & Syntax

```
import java.util.ArrayDeque;
ArrayDeque<Integer> ad = new ArrayDeque<>();
```

ArrayDeque as Stack

Stack Methods

Stack Operation	ArrayDeque Method
Push	`push(x)`
Pop	`pop()`
Peek	`peek()`

Example

```
ArrayDeque<Integer> stack = new ArrayDeque<>();

stack.push(10);
stack.push(20);

System.out.println(stack.peek());
System.out.println(stack.pop());
System.out.println(stack.peek());
```

Output

```
20
20
10
```

Stack Equivalents

Stack Method	Equivalent Deque Method
`push(x)`	`addFirst(x)`
`pop()`	`removeFirst()`
`peek()`	`peekFirst()`

ArrayDeque as Queue

Queue Methods

Non-Throwing	Throwing
`offer(x)`	`add(x)`
`poll()`	`remove()`
`peek()`	`element()`

Example

```
ArrayDeque<Integer> q = new ArrayDeque<>();

q.offer(10);
q.offer(20);

System.out.println(q.peek());
System.out.println(q.poll());
System.out.println(q.peek());
```

Output

```
10
10
20
```

Queue Equivalents

Queue Method	Equivalent Deque Method
`offer()`	`offerLast()`

`add()`	`addLast()`
`poll()`	`pollFirst()`
`remove()`	`removeFirst()`
`peek()`	`peekFirst()`
`element()`	`getFirst()`

ArrayDeque as Deque

Non-Throwing Methods

Operation	Front	Rear
Insert	`offerFirst()`	`offerLast()`
Remove	`pollFirst()`	`pollLast()`
Access	`peekFirst()`	`peekLast()`

Throwing Methods

Operation	Front	Rear
Insert	`addFirst()`	`addLast()`
Remove	`removeFirst()`	`removeLast()`
Access	`getFirst()`	`getLast()`

Example

```
ArrayDeque<Integer> d = new ArrayDeque<>();

d.offerFirst(10);
d.offerLast(20);

d.offerFirst(30);
d.offerLast(40);

System.out.println(d.peekFirst());
System.out.println(d.peekLast());

System.out.println(d.pollFirst());
System.out.println(d.peekFirst());

System.out.println(d.pollLast());
System.out.println(d.peekLast());
```

Output

```
30
40
30
10
```

Method Redundancies

Functionality	Queue	Deque	Stack
Insert Front	—	`addFirst()`	`push()`
Remove Front	`remove()`	`removeFirst()`	`pop()`
Access Front	`element()`	`getFirst()`	`peek()`
Safe Insert	`offer()`	`offerLast()`	—
Safe Remove	`poll()`	`pollFirst()`	—
Safe Access	`peek()`	`peekFirst()`	`peek()`

Traversal
Forward Traversal

```
for(Integer x : ad)
   System.out.print(x + " ");
```

Or

```
Iterator<Integer> it =ad.iterator();
```

Reverse Traversal

```
Iterator<Integer> it = ad.descendingIterator();
```

Why Prefer ArrayDeque?

Compared With	Why ArrayDeque is Better
`Stack`	No synchronization overhead
`Vector`	Not a legacy class
`LinkedList`	Better cache locality
`LinkedList`	Faster average performance

Time Complexity

Category	Methods	Complexity
Stack Operations	`push()`, `pop()`, `peek()`	**O(1) amortized**
Queue Operations	`offer()`, `poll()`, `peek()`	**O(1) amortized**
Deque Insert	`offerFirst()`, `offerLast()`, `addFirst()`, `addLast()`	**O(1) amortized**

Deque Remove	`pollFirst()`, `pollLast()`, `removeFirst()`, `removeLast()`	**O(1) amortized**
Deque Access	`peekFirst()`, `peekLast()`, `getFirst()`, `getLast()`	**O(1)**

Why Amortized O(1)?

`ArrayDeque` uses a **dynamic circular array**.

Most Operations

Insertion/Deletion → O(1)

Occasionally

Array becomes full
↓
Resize occurs
↓
O(n)

Therefore: Average over many operations -> Amortized O(1)

Internal Working

Circular Array

```
   Front
     ↓
[ ][ ][ ][ ][ ]

Rear
↓
```

Uses:

- **Front pointer**
- **Rear pointer**
- **Circular indexing**

This enables:

Insert/Delete at both ends
in O(1) amortized time.

Quick Revision Table

Topic	Remember
Implementation	Resizable Circular Array
Implements	Deque
Can Act As	Stack + Queue + Deque
Preferred Over	Stack, LinkedList

Thread Safe	No
Null Elements	Not Allowed
Random Access	Not Supported
Stack Complexity	O(1) amortized
Queue Complexity	O(1) amortized
Deque Complexity	O(1) amortized
Best Use Case	Single-threaded applications

Interview Notes

// ArrayDeque is the preferred implementation
// for Stack and Queue in single-threaded applications.
// Internally uses a dynamic circular array,
// providing O(1) amortized operations at both ends.